La cola de prioridad (Priority queue): SplPriorityQueue

Caso práctico

El servicio técnico de **BK Programación** recibe incidencias continuamente. Hasta ahora se atendían por orden de llegada, y el día que se cayó el servidor de la tienda estuvo dos horas esperando detrás de "cambiar el tóner". **Juan** quiere que se atienda siempre primero la incidencia **más grave**, aunque haya llegado la última. **María** le dice que eso es una **cola de prioridad**.

1.- Qué es una cola de prioridad

Una **cola de prioridad** es un ADT en el que cada elemento tiene una **prioridad** y **siempre sale el de mayor prioridad**, independientemente de cuándo llegó. Sus operaciones son:

Operación	Qué hace	En SplPriorityQueue	Coste
insert	Añadir un elemento con su prioridad	`insert($valor, $prioridad)`	O(log n)
extract	Sacar el de mayor prioridad	`extract()`	O(log n)
peek	Ver el de mayor prioridad sin sacarlo	`top()`	O(1)
size, *isEmpty*	Tamaño / ¿vacía?	`count($cola)`, `isEmpty()`	O(1)

Por dentro, `SplPriorityQueue` es un **montículo** (*heap*): un árbol binario guardado en un array en el que cada nodo tiene más prioridad que sus hijos. El de mayor prioridad está siempre en la raíz, y al insertar o extraer solo hay que reordenar un camino del árbol (de altura log n). Por eso, con un millón de elementos, cada operación necesita unos 20 pasos.

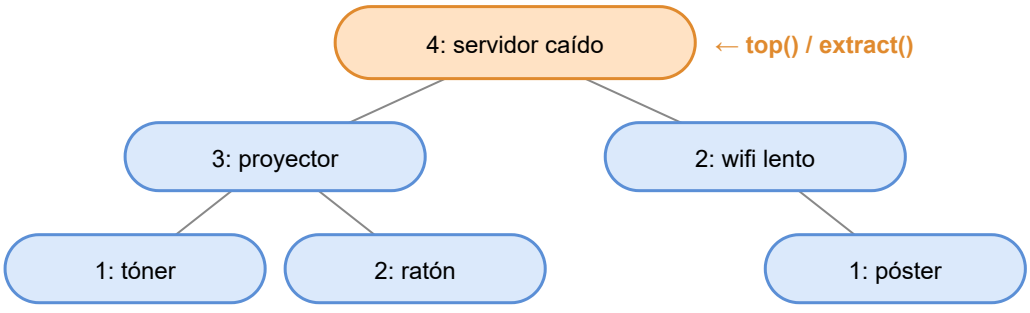


Figura 1. El montículo interno de una cola de prioridad: la raíz es la de mayor prioridad

2.- La clase SplPriorityQueue

```php
<?php
$incidencias = new SplPriorityQueue();
$incidencias->insert('Cambiar el tóner', 1);
$incidencias->insert('El servidor no responde', 4);
$incidencias->insert('Wifi lento en el aula 12', 2);

echo $incidencias->top();         // "El servidor no responde" (solo mirar)
echo $incidencias->extract();     // "El servidor no responde"
echo $incidencias->extract();     // "Wifi lento en el aula 12"
```

- La prioridad puede ser cualquier valor que PHP sepa comparar: números, textos ("2026-10-20"), **arrays** (se comparan elemento a elemento)...
- `setExtractFlags()` indica qué devuelven `extract()` y `top()`: `SplPriorityQueue::EXTR_DATA` (el valor, por defecto), `EXTR_PRIORITY` (la prioridad) o `EXTR_BOTH` (un array `['data' => ..., 'priority' => ...]`).
- Con la cola vacía, `extract()` y `top()` lanzan `RuntimeException`.

Debes conocer

Con prioridades iguales, el orden NO está garantizado. `SplPriorityQueue` no respeta el orden de llegada entre elementos con la misma prioridad (en el ejemplo `e01`, de tres pacientes con gravedad 3, salió antes Eva que Carmen, aunque llegó después). Si el orden de llegada importa, usa como prioridad un array `[prioridad, -orden]`: a igual prioridad, gana el número de orden menor.

```php
<?php
foreach ($llegadas as $orden => [$paciente, $gravedad]) {
    $urgencias->insert($paciente, [$gravedad, -$orden]);    // [3, -1] > [3, -2]: antes el
que llegó antes
}
```

Debes conocer

Recorrer una cola de prioridad la vacía. `foreach ($cola as $x)` va **extrayendo** los elementos, igual que un bucle de `extract()`. Si solo quieres verlos, recorre una **copia**: `foreach (clone $cola as $x)`.

3.- Cambiar el criterio: compare()

Por defecto sale la **mayor** prioridad. Para que salga la **menor** (por ejemplo, la fecha más próxima), hay dos opciones:

- insertar la prioridad con el signo cambiado (`-$minutos`, `-strtotime($fecha)`), o
- heredar de `SplPriorityQueue` y redefinir `compare($prioridad1, $prioridad2)`, que debe devolver un número **positivo si `$prioridad1` tiene más prioridad** que `$prioridad2`:

```
<?php
final class ColaPorHora extends SplPriorityQueue
{
    public function compare(mixed $prioridad1, mixed $prioridad2): int
    {
        return $prioridad2 <=> $prioridad1;     // al revés: la hora MENOR sale antes
    }
}
$recogidas = new ColaPorHora();
$recogidas->insert('pedido 1001', '18:30');
$recogidas->insert('pedido 1002', '13:15');
echo $recogidas->extract();                     // "pedido 1002"
```

4.- Aplicaciones típicas

Problema	Prioridad
Incidencias, triaje de urgencias, embarque por grupos	La gravedad o el grupo (y el orden de llegada para desempatar)
Planificadores de tareas, mensajes programados	La fecha (la más próxima primero)
Simulaciones por eventos	El instante en que ocurre cada evento
Camino más corto en un mapa con distancias (algoritmo de Dijkstra)	La distancia acumulada (la menor primero)
Los k elementos mayores de un flujo de datos	El valor

El algoritmo de Dijkstra calcula la distancia más corta desde un origen a todos los demás puntos de una red con distancias (carreteras, líneas de autobús, una red de ordenadores). La cola de prioridad entrega siempre el punto pendiente más cercano al origen y, desde él, se mejoran las distancias de sus vecinos. Si se usara una cola normal (búsqueda en anchura), se encontraría el camino con **menos tramos**, que no tiene por qué ser el más corto.

5.- Ejercicio resuelto: Dijkstra con las carreteras gallegas

Ejercicio resuelto

Calcula la distancia más corta desde A Coruña a cada ciudad y la ruta a Vigo.

```
<?php
$carreteras = [
    'A Coruña' => ['Santiago' => 73, 'Lugo' => 97, 'Ferrol' => 55],
    'Ferrol' => ['A Coruña' => 55, 'Lugo' => 114],
```

```
    'Santiago' => ['A Coruña' => 73, 'Pontevedra' => 62, 'Lugo' => 106, 'Ourense' => 111],
    'Lugo' => ['A Coruña' => 97, 'Ferrol' => 114, 'Santiago' => 106, 'Ourense' => 96],
    'Pontevedra' => ['Santiago' => 62, 'Vigo' => 27, 'Ourense' => 100],
    'Vigo' => ['Pontevedra' => 27, 'Ourense' => 101],
    'Ourense' => ['Santiago' => 111, 'Lugo' => 96, 'Pontevedra' => 100, 'Vigo' => 101],
];
function dijkstra(array $grafo, string $origen): array
{
    $distancia = [$origen => 0];
    $anterior = [];
    $cola = new SplPriorityQueue();
    $cola->setExtractFlags(SplPriorityQueue::EXTR_BOTH);
    $cola->insert($origen, 0);
    while (!$cola->isEmpty()) {
        ['data' => $ciudad, 'priority' => $p] = $cola->extract();   // la más cercana
pendiente
        if (-$p > $distancia[$ciudad]) {
            continue;                                               // entrada antigua:
ya hay una mejor
        }
        foreach ($grafo[$ciudad] as $vecina => $km) {
            $nueva = $distancia[$ciudad] + $km;
            if ($nueva < ($distancia[$vecina] ?? PHP_INT_MAX)) {
                $distancia[$vecina] = $nueva;
                $anterior[$vecina] = $ciudad;
                $cola->insert($vecina, -$nueva);                    // -distancia: la
menor sale antes
            }
        }
    }
    return [$distancia, $anterior];
}
[$km, $anterior] = dijkstra($carreteras, 'A Coruña');
// Ferrol 55, Santiago 73, Lugo 97, Pontevedra 135, Vigo 162, Ourense 184
$ruta = ['Vigo'];
while (isset($anterior[$ruta[0]])) {
    array_unshift($ruta, $anterior[$ruta[0]]);
}
echo implode(' -> ', $ruta);     // A Coruña -> Santiago -> Pontevedra -> Vigo
```

Observa que `SplPriorityQueue` no permite "cambiar la prioridad" de un elemento que ya está dentro. Por eso, cuando se mejora la distancia a una ciudad, se inserta **otra vez** con la nueva distancia, y las entradas antiguas se descartan al salir (`continue`).

Autoevaluación

- Se insertan 'a' (prioridad 1), 'b' (3) y 'c' (2). ¿Qué devuelve `extract()`? **"b".**
- ¿Qué cuesta insertar en una cola de prioridad de n elementos? **O(log n).**
- Dos elementos tienen la misma prioridad. ¿Sale antes el que se insertó antes? **No está garantizado: hay que desempatar con `[prioridad, -orden]`.**
- ¿Cómo consultas los elementos de una `SplPriorityQueue` sin vaciarla? **Recorriendo una copia: `foreach (clone $cola as ...)`.**
- ¿Qué devuelve `extract()` después de `setExtractFlags(SplPriorityQueue::EXTR_BOTH)`? **Un array con `data` y `priority`.**

6.- Ejercicios

Fichero	Contenido
`Ejercicios PriorityQueue/e01-SplPriorityQueue.php`	La API: `insert`, `extract`, `top`, `setExtractFlags`, prioridades iguales y desempate, `foreach` sobre una copia, `compare()`
`Ejercicios PriorityQueue/e02-SplPriorityQueue.php`	Aplicaciones: triaje de urgencias, mensajes programados, Dijkstra
`Ejercicios PriorityQueue/problemas/`	`e01` y `e02`: problemas y soluciones